

Lý thuyết chuyên sâu: Đệ quy & Thuật toán Quay lui (Recursion & Backtracking)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Bản chất của Đệ quy (Recursion)

1.1. Khái niệm & Cơ chế Call Stack

Đệ quy là kỹ thuật mà một hàm tự gọi lại chính nó với một tập tham số nhỏ hơn, đưa bài toán phức tạp về các bài toán con tương tự nhưng có kích thước nhỏ hơn cho đến khi đạt được **trường hợp cơ sở (Base Case)** có thể giải trực tiếp.

Mỗi khi một hàm được gọi, hệ điều hành sẽ cấp phát một **khung ngăn xếp (Stack Frame)** trên bộ nhớ Call Stack của chương trình:

- **Nội dung Stack Frame:**
 1. Các tham số truyền vào hàm.
 2. Các biến cục bộ của hàm.
 3. Địa chỉ trả về (Return Address) để CPU quay lại thực thi lệnh tiếp theo khi hàm kết thúc.
- **Kích thước bộ nhớ:** Độ sâu đệ quy lớn nhất là D thì bộ nhớ Call Stack tốn $O(D \times S)$, trong đó S là kích thước của mỗi khung ngăn xếp.

 **Hiện tượng tràn bộ nhớ ngăn xếp (Stack Overflow):**

- Trên Windows, giới hạn mặc định của ngăn xếp (Stack Size) thường chỉ là 1MB đến 8MB.
- Trong các kỳ thi ICPC / OLP (môi trường Linux), giới hạn ngăn xếp thường bằng bộ nhớ chương trình (256MB hoặc 512MB).
- Nếu hàm đệ quy có độ sâu vượt quá 10^6 lần gọi hoặc tạo ra mảng cục bộ kích thước lớn trong hàm, chương trình sẽ lập tức bị lỗi `Segmentation Fault` (SIGSEGV / Exit code khác 0).

1.2. Phân loại các dạng đệ quy

1. **Đệ quy tuyến tính (Linear Recursion):** Hàm chỉ gọi lại chính nó đúng một lần tại mỗi bước (ví dụ: Tính giai thừa $n! = n \times (n - 1)!$, tìm kiếm nhị phân).
2. **Đệ quy nhị phân / Cây (Tree Recursion):** Hàm gọi lại chính nó hai hoặc nhiều lần (ví dụ: Dãy Fibonacci nguyên sơ $F(n) = F(n - 1) + F(n - 2)$, thuật toán Merge Sort).

3. **Đệ quy đuôi (Tail Recursion):** Lời gọi đệ quy là thao tác cuối cùng được thực thi trong hàm trước khi `return`. Trình biên dịch hỗ trợ **TCO (Tail Call Optimization)** có thể tối ưu hàm này thành vòng lặp, giữ cho độ sâu Stack chỉ là $O(1)$.
4. **Đệ quy hỗ tương (Mutual / Indirect Recursion):** Hàm A gọi hàm B, hàm B lại gọi hàm A.
-

1.3. Đánh giá độ phức tạp thời gian của hàm đệ quy

A. Phương pháp Cây đệ quy (Recursion Tree)

Mô hình hóa toàn bộ các lời gọi hàm dưới dạng một cây có gốc là lời gọi ban đầu. Tổng thời gian chạy là tổng khối lượng công việc trên tất cả các nút của cây:

$$\text{Time} = \sum_{\text{level } l} (\text{Số nút ở tầng } l \times \text{Chi phí tại mỗi nút})$$

Ví dụ: Hàm Fibonacci ngây thơ $F(n) = F(n-1) + F(n-2)$:

- Cây nhị phân đầy đủ có độ sâu n .
- Số nút tối đa ở tầng l là 2^l .
- Tổng số nút $\approx 2^0 + 2^1 + \dots + 2^n \approx O(2^n)$.

B. Định lý Thợ (Master Theorem)

Áp dụng cho các thuật toán đệ quy chia để trị có dạng hệ thức truy hồi:

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

Trong đó $a \geq 1$ (số bài toán con), $b > 1$ (tỷ lệ thu nhỏ kích thước), $d \geq 0$ (chi phí phân chia và gộp kết quả):

1. Nếu $\log_b a > d \implies T(n) = O(n^{\log_b a})$ (Chi phí tầng lá chiếm ưu thế).
 2. Nếu $\log_b a = d \implies T(n) = O(n^d \log n)$ (Chi phí các tầng cân bằng, ví dụ Merge Sort: $a = 2, b = 2, d = 1 \implies O(n \log n)$).
 3. Nếu $\log_b a < d \implies T(n) = O(n^d)$ (Chi phí tại nút gốc chiếm ưu thế).
-

2. Thuật toán Quay lui (Backtracking)

2.1. Bản chất & Cây không gian trạng thái (State Space Tree)

Thuật toán Quay lui (Backtracking) là một kỹ thuật thiết kế giải thuật dùng để giải các bài toán tìm kiếm tổ hợp (liệt kê, đếm, hoặc tối ưu hóa) bằng cách xây dựng dần từng thành phần của nghiệm ứng viên.

- **Cây không gian trạng thái (State Space Tree):**
 - Nút gốc (Root): Trạng thái rỗng (chưa chọn phần tử nào).
 - Tầng i (Level i): Các trạng thái sau khi đã đưa ra quyết định cho biến thứ i .
 - Nút lá (Leaf): Một trạng thái hoàn chỉnh (nghiệm tiềm năng hoặc không hợp lệ).

- **Bản chất hoạt động:** Thực chất là một phép **Duyệt theo chiều sâu (DFS - Depth First Search)** trên cây không gian trạng thái ẩn.

2.2. Bộ ba thao tác kinh điển: Choose → Explore → Unchoose

Mọi thuật toán quay lui đều xoay quanh một khung sườn bất biến:

```
void backtrack(trạng thái hiện tại) {
    if (đạt đến điều kiện dừng - hoàn thành nghiệm) {
        Ghi nhận nghiệm / Cập nhật kết quả tối ưu;
        return;
    }

    for (mỗi lựa chọn hợp lệ có thể đi tiếp) {
        1. CHOOSE:    Thực hiện lựa chọn, đánh dấu / cập nhật trạng thái;
        2. EXPLORE:   Gọi đệ quy bước tiếp theo backtrack(trạng thái mới);
        3. UNCHOOSE:  Hoàn tác lựa chọn, phục hồi trạng thái như trước khi chọn (BACKTRACK);
    }
}
```

Nguyên tắc "Đi đến đâu, dọn dẹp đến đó":

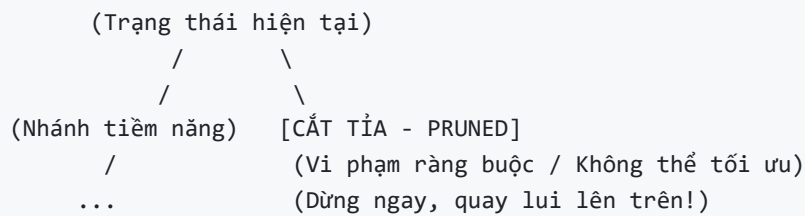
Thao tác **Unchoose** (khôi phục trạng thái) là linh hồn của thuật toán quay lui. Nếu quên thao tác này, trạng thái của nhánh duyệt trước sẽ làm hỏng các nhánh duyệt sau!

2.3. Các không gian tìm kiếm tiêu biểu trong CP

Dạng bài toán	Kích thước không gian	Giới hạn N an toàn trong CP
Sinh tập con / Xâu nhị phân ($x_i \in \{0, 1\}$)	$O(2^N)$	$N \leq 20 \sim 25$
Sinh tổ hợp chập K	$O\left(\binom{N}{K}\right)$	$\binom{N}{K} \leq 10^7$
Sinh hoán vị (Mỗi phần tử dùng đúng 1 lần)	$O(N!)$	$N \leq 10 \sim 11$ ($11! \approx 3.9 \times 10^7$)
Xếp N quân hậu (N -Queens)	$\ll O(N!)$ (nhờ cắt tỉa)	$N \leq 14 \sim 16$
Phân hoạch số nguyên (Integer Partition)	$P(N) \sim \frac{1}{4N\sqrt{3}} e^{\pi\sqrt{2N/3}}$	$N \leq 60 \sim 70$

3. Kỹ thuật Cắt tỉa nhánh cận (Pruning & Branch and Bound)

Khi không gian tìm kiếm thuần túy là $O(2^N)$ hoặc $O(N!)$, bài toán sẽ bị TLE nếu duyệt ngây thơ. **Cắt tỉa (Pruning)** là kỹ thuật nhận diện sớm các nhánh cây **không thể dẫn tới nghiệm hợp lệ hoặc nghiệm tối ưu** để quay lui ngay lập tức mà không đi sâu thêm.



3.1. Cắt tỉa tính khả thi (Feasibility Pruning)

Cắt ngay khi một ràng buộc của bài toán bị vi phạm, ví dụ:

- Trong bài toán tổng tập con (Subset Sum): Nếu tổng các phần tử đang chọn $S > \text{Target}$ và các số đều dương $\implies$ Ngắt ngay nhánh này.
- Trong bài toán N -Queens: Nếu vị trí (row, col) nằm trên cùng cột, đường chéo chính hoặc đường chéo phụ với một quân hậu đã đặt trước đó $\implies$ Không bao giờ gọi đệ quy vào ô này.

3.2. Cắt tỉa tính tối ưu / Nhánh cận (Optimality Pruning / Branch and Bound)

Áp dụng cho các bài toán **tìm giá trị nhỏ nhất / lớn nhất**:

- Giả sử ta đang tìm nghiệm có chi phí nhỏ nhất và đã tìm được nghiệm tốt nhất hiện tại là `best_ans`.
- Tại trạng thái hiện tại u , ta đã tốn chi phí thực tế là $g(u)$.
- Ta ước lượng chi phí tối thiểu để đi từ u đến đích là $h(u)$ (Cận dưới - Lower Bound).
- **Quy tắc cắt tỉa:**

$$\text{Nếu } g(u) + h(u) \geq \text{best_ans} \implies \text{Cắt tỉa ngay!}$$

Vì dù có đi tiếp một cách tối ưu nhất trong nhánh này thì kết quả cũng không thể vượt qua `best_ans`.

3.3. Tối ưu thứ tự duyệt (Search Order & Fail-First Principle)

Thứ tự duyệt các nhánh ảnh hưởng khổng lồ đến hiệu quả cắt tỉa:

- **Nguyên tắc "Thất bại sớm" (Fail-First / MRV - Minimum Remaining Values):** Luôn ưu tiên đưa ra quyết định ở biến có **ít lựa chọn hợp lệ nhất**. Việc này giúp cây tìm kiếm phát hiện sai lầm sớm ở các tầng nông thay vì lãng phí thời gian ở tầng sâu (cực kỳ hiệu quả trong bài toán Sudoku).
- **Sắp xếp theo kỳ vọng:** Trong bài toán tối ưu, hãy thử các lựa chọn có khả năng cho chi phí tốt trước để cập nhật `best_ans` thật sớm, từ đó cắt tỉa được nhiều nhánh phía sau hơn.

4. Tối ưu hóa bằng Phép toán Bit (Bitwise Backtracking)

Trong C++, thay vì sử dụng mảng boolean `bool used[N]` để đánh dấu các phần tử đã thăm:

1. Dùng một số nguyên không dấu (`int` cho $N \leq 30$, `unsigned long long` cho $N \leq 64$) làm **mặt nạ bit (bitmask)**.
2. Thao tác trên bit chạy trực tiếp trên thanh ghi CPU với thời gian $O(1)$ và nhanh gấp 5 – 10 lần mảng phụ:
 - **Kiểm tra phần tử i đã dùng chưa:** `(mask >> i) & 1`
 - **Đánh dấu phần tử i :** `mask | (1 << i)`

- Bỏ đánh dấu phần tử i : `mask ^ (1 << i)`
- Lấy ra bit 1 thấp nhất (Lowest Set Bit): `mask & (-mask)`

5. Kỹ thuật Meet-in-the-middle (Chia đôi không gian tìm kiếm)

Khi $N \approx 40$, độ phức tạp $O(2^{40}) \approx 1.1 \times 10^{12}$ là hoàn toàn bất khả thi trong thời gian 1.0s.

Tuy nhiên, nếu ta chia tập N phần tử thành 2 nửa bằng nhau ($N_1 = 20, N_2 = 20$):

1. Nửa thứ nhất: Sinh tất cả các trạng thái trong $O(2^{N/2}) = 2^{20} \approx 10^6$, lưu vào mảng `A` và sắp xếp lại.
2. Nửa thứ hai: Sinh tất cả các trạng thái trong $O(2^{N/2})$, với mỗi trạng thái, dùng **Tìm kiếm nhị phân** (`std::lower_bound`) hoặc **Hai con trỏ** trên mảng `A` để tìm phần ghép tương ứng trong $O(\log(2^{N/2})) = O(N/2)$.
3. **Tổng độ phức tạp:**

$$O(2^{N/2} \log(2^{N/2})) = O(N \cdot 2^{N/2})$$

Với $N = 40$, $40 \cdot 2^{20} \approx 4 \times 10^7$ phép tính, **chạy mượt mà trong dưới 0.2 giây!**

6. Tài liệu tham khảo & Bài giảng đề xuất (References)

6.1. Giáo trình & Sách thuật toán kinh điển

1. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
2. [Chapter 5: Complete Search \(Generating Subsets, Permutations, Backtracking, Pruning, Meet-in-the-middle\)](#).
3. **Introduction to Algorithms (CLRS 4th Edition)** — *Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein*:
4. *Chapter 4: Divide-and-Conquer, Recurrences and Master Theorem.*
5. *Chapter 34: NP-Completeness and Exhaustive Search.*
6. **Competitive Programming 4 (CP4 - Book 1)** — *Steven Halim, Felix Halim, Suhendry Effendy*:
7. *Section 3.2: Complete Search (Iterative, Recursive Backtracking, Tips to Speed Up).*
8. **Giáo trình Chuyên đề Giải thuật** — *Thầy Lê Minh Hoàng (Đại học Sư phạm Hà Nội)*:
9. *Chương: Thuật toán Quay lui (Backtracking)* — Phân tích bài toán Mã đi tuần, N -Queens và phương pháp cắt nhánh tìm kiếm.
10. **Tài liệu Chuyên Tin (Tập 1)** — *Thầy Hồ Sĩ Đàm (Chủ biên)*:
11. *Chuyên đề: Đề quy và thuật toán quay lui trong hình học và tổ hợp.*

6.2. Bài giảng & Chuyên đề trực tuyến (Online Courses & Wikis)

1. **VNOI Wiki**:
2. [Thuật toán quay lui \(Backtracking\)](#) — Cẩm nang toàn diện về mô hình quay lui và cắt tỉa nhánh cận.
3. [Kỹ thuật Meet-in-the-middle](#) — Ứng dụng chia đôi không gian tìm kiếm trong ICPC.
4. [Độ phức tạp thuật toán và Đề quy](#) — Phân tích cây đệ quy và định lý Master.

5. **Stanford University (CS106B - Programming Abstractions):**
 6. [Lecture Notes: Recursive Backtracking](#) (Julie Zelenski & Keith Schwarz) — Bài giảng trực quan hóa kinh điển về mô hình *Choose - Explore - Unchoose*, giải đố Sudoku và mê cung.
 7. **MIT OpenCourseWare (6.006 - Introduction to Algorithms):**
 8. [Lecture 11: Dynamic Programming & Recursion DAGs](#) — So sánh đệ quy nhánh quay lui và quy hoạch động tối ưu hóa trạng thái.
 9. **CP-Algorithms:**
 10. [Search Algorithms & Submask Enumeration](#) — Duyệt các tập con bằng bitmask tối ưu.
-
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.